

Transformation et manipulation de données



Dr Khadidja Henni

Séance 9: Classes non balancées

Plan de cours

- Introduction
- Métriques d'évaluation
- Intervenir sur les données (under/over-sampling, augmentation).
- Intervenir sur l'algorithme (pondération des pertes, ensembles spécialisés, approches deep learning).
- Laboratoire formatif

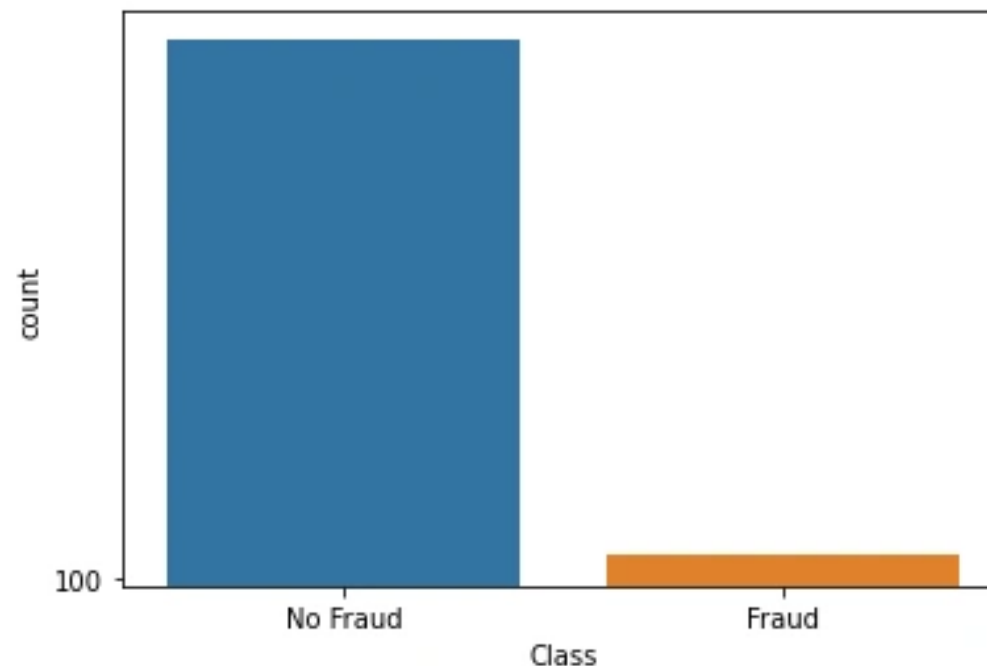
Introduction

- Dans les jeux de données réels, les classes sont rarement équilibrées. La classe majoritaire domine l'entraînement et biaise la fonction d'optimisation vers une précision globale élevée au détriment des cas rares.
- Or ces cas rares sont souvent les plus importants (fraude, maladie, défaut).
- Résultat : le modèle peut « bien » performer en moyenne mais échouer là où cela compte.

Définition du déséquilibre

- Un dataset est déséquilibré quand la proportion d'exemples d'une classe dépasse largement celle d'une autre (ex. 99/1, 95/5, 90/10).
 - Classe majoritaire = catégorie la plus fréquente;
 - minoritaire = catégorie rare.

Le ratio n'est qu'un indicateur: l'impact dépend aussi de la séparation des distributions et des coûts d'erreur.



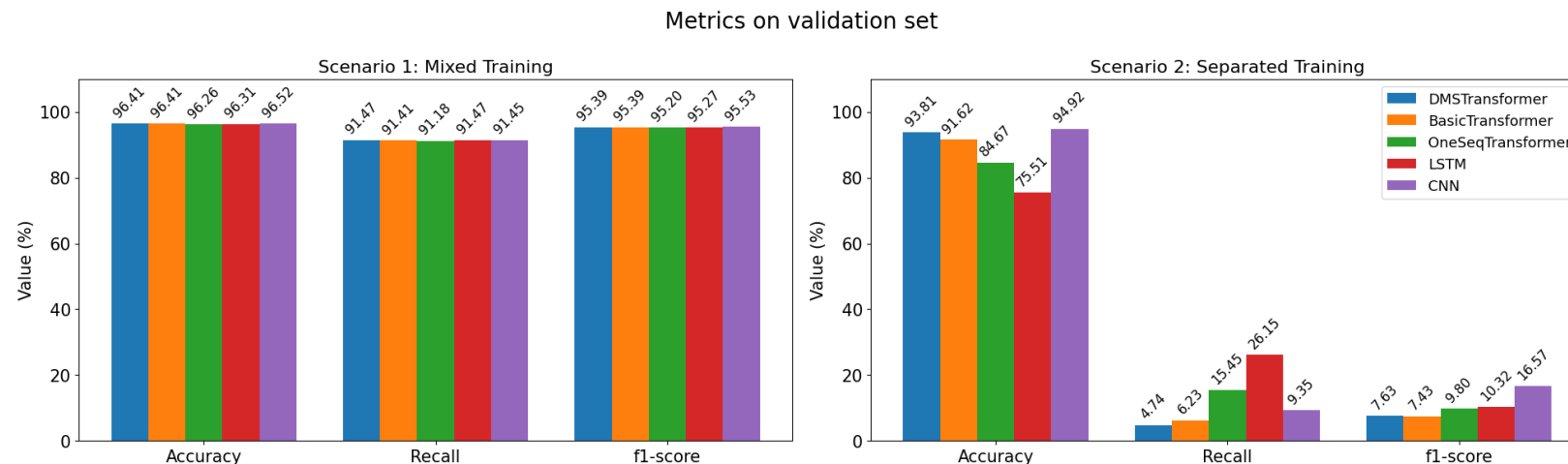
Exemple

- **Contexte** : Détection de fraudes dans une entreprise fintech.
- **Modèle** : Algorithme de classification avec **98 % de précision** annoncée.
- **Test réel** : Une transaction frauduleuse et une non frauduleuse → toutes deux prédites comme **non frauduleuses**.
- **Problème identifié** :
 - Jeu de données **déséquilibré** : 98 % non frauduleuses vs 2 % frauduleuses.
 - La métrique utilisée (**accuracy**) favorise la majorité.
- **Conclusion** : Le modèle prédit toujours "non frauduleuse", ce qui donne artificiellement 98 % de précision mais aucune capacité réelle à détecter la fraude

Conséquences du déséquilibre

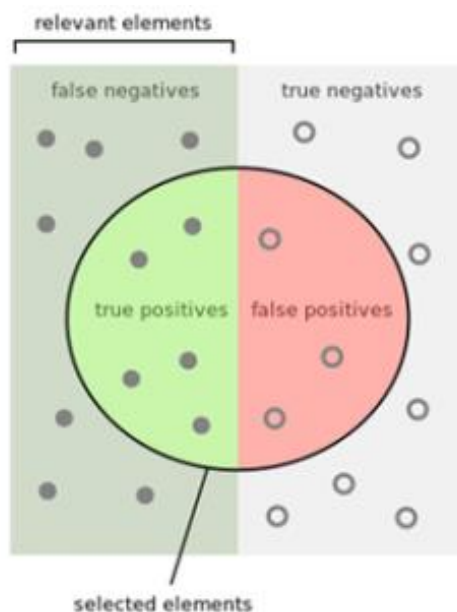
- Rappel (Recall) de la classe minoritaire très faible (beaucoup de FN).
- Illusion de performance via l'accuracy.
- Coût réel élevé : fraude non détectée, patient non diagnostiqué, panne non anticipée.

L'objectif est de déplacer l'optimum d'entraînement vers une meilleure sensibilité aux cas rares.



Limites de l'accuracy

- L'accuracy punit peu les erreurs sur la minorité car la majorité domine.
- Un objectif pertinent doit valoriser les vrais positifs de la classe rare et pénaliser les faux négatifs.
- On préférera des métriques comme le F1, le Recall, le ROC-AUC et surtout le PR-AUC lorsque la rareté est extrême.



How many selected items are relevant?

$$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

How many relevant items are selected?

$$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

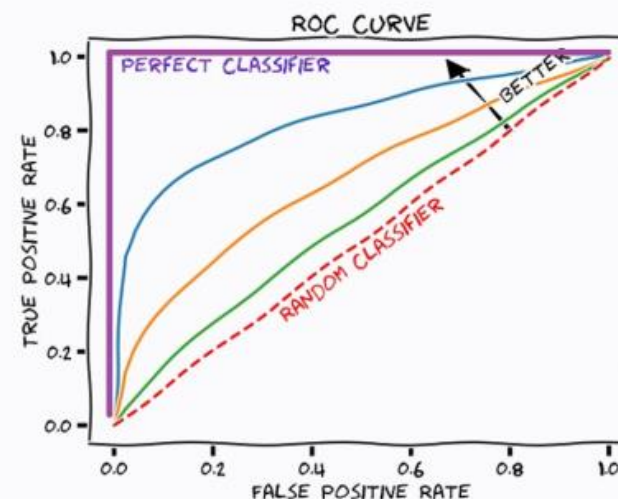
Métriques adaptées

- Matrice de confusion (TP, FP, TN, FN).
- Précision (Precision) = $TP / (TP + FP)$: fiabilité des positifs prédits.
- Rappel (Recall) = $TP / (TP + FN)$: capacité à capter les vrais positifs.
- F1 = $2 \cdot (Prec \cdot Rec) / (Prec + Rec)$: compromis utile lorsque les classes sont déséquilibrées.

Métriques adaptées

1. ROC-AUC (Receiver Operating Characteristic – Area Under Curve)

- **Définition** : mesure la capacité globale d'un modèle à distinguer positifs et négatifs, en traçant le compromis entre **taux de vrais positifs (rappel)** et **taux de faux positifs**.
- **Interprétation** : plus l'AUC est proche de 1, meilleure est la séparation.
- **Limite** : quand les classes sont très déséquilibrées (ex. fraude = 2 %), un modèle peut avoir une **ROC-AUC élevé** même s'il est mauvais pour détecter les rares positifs.



ROC
RECEIVER OPERATING
CHARACTERISTIC

AUC
AREA UNDER THE CURVE

Métriques adaptées

2. PR-AUC (Precision–Recall Area Under Curve)

- **Définition** : se concentre uniquement sur la performance de la classe positive (fraude).
- **Trace** : le compromis entre **précision (prédictions positives correctes)** et **rappel (capacité à trouver tous les positifs)**.
- **Avantage** : plus sensible et plus représentatif quand la classe positive est **rare**.
- **Exemple** : utile pour détecter la fraude, les maladies rares, etc.

Métriques adaptées

- **3. Coût attendu (Cost-sensitive learning)**
- **Idée** : toutes les erreurs n'ont pas le même impact.
- **Exemple** :
 - Faux négatif (ne pas détecter une fraude) → perte financière énorme.
 - Faux positif (bloquer une transaction légitime) → coût plus faible mais gêne l'utilisateur.
- **Utilité** : on définit un **coût métier** pour chaque type d'erreur, puis on optimise le modèle pour **minimiser le coût total attendu** plutôt que la simple précision

Résumé du problème

- Le déséquilibre exige d'adapter à la fois **l'ENTRAÎNEMENT** (données/algorithmes) et **l'ÉVALUATION** (métriques/seuils).
- On combine souvent plusieurs leviers : rééchantillonnage, pondération, ajustement de seuil, et validation rigoureuse (CV stratifiée).

Solution

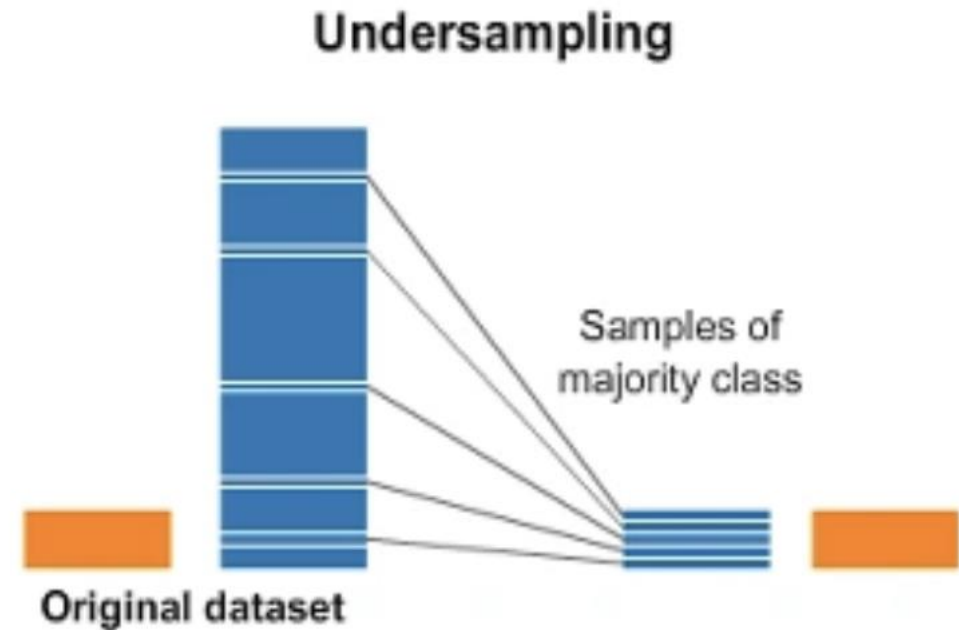
Deux familles:

1. Intervenir sur les données (under/over-sampling, augmentation).
2. Intervenir sur l'algorithme (pondération des pertes, ensembles spécialisés, approches deep learning).

On verra aussi des approches hybrides mêlant ces idées.

Sous-échantillonnage (Undersampling)

- **Principe:** réduire la classe majoritaire pour équilibrer le dataset.
- **Avantages:** rapidité, training plus court, équilibre immédiat.
- **Limites:** perte d'information (variabilité de la majorité), variance accrue.
- **Variantes:** RandomUnderSampler, ClusterCentroids (remplace la majorité par ses centroïdes).



Sous-échantillonnage (Undersampling)

a) Random Undersampling

- On sélectionne aléatoirement un sous-ensemble de la classe majoritaire.
- Exemple : 98 000 non-fraudes + 2 000 fraudes → on réduit à 2 000 non-fraudes + 2 000 fraudes.

Avantage : rapide, simple.

Inconvénient : perte d'information (on jette beaucoup de données).

b) Undersampling informé

- Au lieu de supprimer au hasard, on choisit **les exemples les plus représentatifs** ou **les plus proches de la frontière de décision**.
- Méthodes comme **NearMiss** (garde les exemples majoritaires les plus proches des minoritaires).
- **Avantage** : on garde les données « utiles ».
- **Inconvénient** : plus coûteux à calculer.

Sous-échantillonnage (Undersampling)

```
from imblearn.under_sampling import RandomUnderSampler, NearMiss
import pandas as pd

# .... Exemple : X = features, y = labels (0 = non fraude, 1 = fraude)

# Random undersampling
rus = RandomUnderSampler(sampling_strategy=1.0, random_state=42)
X_res, y_res = rus.fit_resample(X, y)

print("Avant undersampling :", pd.Series(y).value_counts())
print("Après undersampling :", pd.Series(y_res).value_counts())

# NearMiss (undersampling informé)
nm = NearMiss()
X_res_nm, y_res_nm = nm.fit_resample(X, y)
```

Sous-échantillonnage (Undersampling)

c) Cluster-based Undersampling

- Plutôt que de supprimer aléatoirement des exemples de la classe majoritaire (RandomUnderSampler), on regroupe ces exemples en **clusters** (souvent via K-Means).
- Chaque cluster est ensuite représenté par son **centroïde** → cela réduit la taille de la classe majoritaire tout en **gardant sa diversité**.
- Ça évite de perdre trop d'information par suppression aléatoire.

Avantage : meilleure représentativité que le hasard.

```
from imblearn.under_sampling import ClusterCentroids
from sklearn.datasets import make_classification
from collections import Counter
import pandas as pd

# 1) Génération d'un dataset déséquilibré
X, y = make_classification(
    n_samples=10000, n_features=10, n_informative=5, n_redundant=2,
    weights=[0.9, 0.1], random_state=42
)

print("Distribution initiale :", Counter(y))

# 2) Cluster Centroids Undersampling
cc = ClusterCentroids(random_state=42, sampling_strategy=1.0)
X_res, y_res = cc.fit_resample(X, y)

print("Après ClusterCentroids :", Counter(y_res))
```

```
Distribution initiale : Counter({np.int64(0): 8961, np.int64(1): 1039})
Après ClusterCentroids : Counter({np.int64(0): 1039, np.int64(1): 1039})
```


Sur-échantillonnage (Oversampling)

- **Principe:** augmenter la minorité.
- **Oversampling** = augmenter artificiellement la taille de la classe minoritaire afin d'équilibrer le dataset.
- **Méthodes principales:**
 - **Random oversampling** : copies exactes → simple mais risque d'overfitting.
 - **SMOTE** : interpolation → diversité accrue.
 - **ADASYN** : génération ciblée sur zones complexes → plus adapté mais plus risqué.

Random Oversampling

- **Principe** : on **équilibre** les **classes** en **dupliquant aléatoirement** les exemples de la classe minoritaire jusqu'à atteindre la taille de la classe majoritaire.
- **Exemple** :
 - Données : 9000 non-fraude (majoritaire), 1000 fraude (minoritaire).
 - Après random oversampling : 9000 non-fraude + 9000 fraude (par duplication).
- **Avantage** : très simple, pas de données artificielles « inventées ».
- **Inconvénient** : risque de **surapprentissage** car le modèle revoit plusieurs fois les mêmes exemples.

```
# Installer si besoin : pip install imbalanced-learn pandas scikit-learn

from imblearn.over_sampling import RandomOverSampler
from sklearn.datasets import make_classification
import pandas as pd
from collections import Counter

# 1) Créons un dataset déséquilibré
X, y = make_classification(
    n_samples=1000, n_features=10, n_informative=5, n_redundant=2,
    weights=[0.9, 0.1], random_state=42
)

print("Avant oversampling :", Counter(y))

# 2) Appliquer le Random Oversampling
ros = RandomOverSampler(sampling_strategy=1.0, random_state=42)
X_res, y_res = ros.fit_resample(X, y)

print("Après oversampling :", Counter(y_res))
```

```
Avant oversampling : Counter({np.int64(0): 896, np.int64(1): 104})
Après oversampling : Counter({np.int64(0): 896, np.int64(1): 896})
```

SMOTE

- **SMOTE (Synthetic Minority Oversampling Technique)**

Principe: Contrairement au **Random Oversampling** (qui fait juste des copies de la classe minoritaire), **SMOTE génère de nouveaux exemples synthétiques**. Il ne duplique pas, mais **interpole** entre des échantillons existants de la minorité.

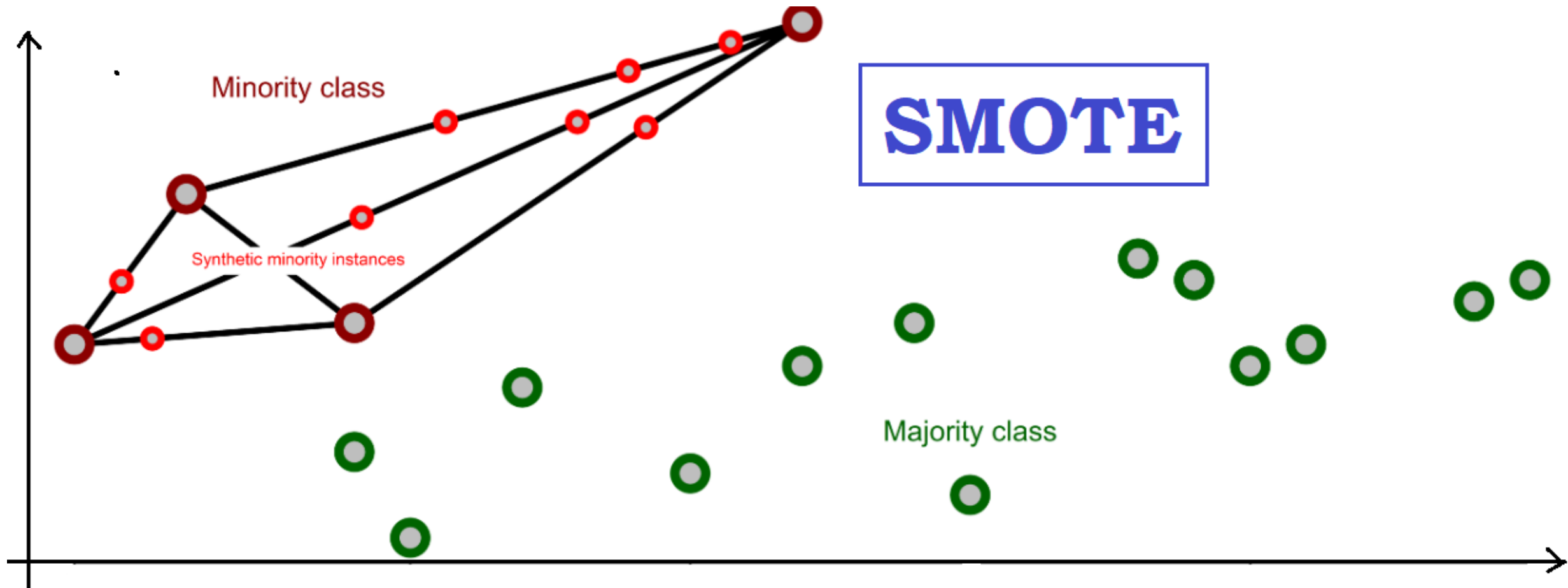
1. **Choisir un point minoritaire** (ex: une transaction frauduleuse).
2. **Trouver ses k plus proches voisins** (dans la minorité, typiquement $k = 5$).
3. **Sélectionner un voisin** aléatoirement parmi ces k .
4. **Créer un point synthétique** :
 - On trace une droite entre le point choisi et son voisin.
 - On place un nouveau point **quelque part sur cette droite** (interpolation aléatoire).

Formule simplifiée :

$$x_{new} = x_i + \lambda \cdot (x_{voisin} - x_i), \lambda \in [0, 1]$$

Résultat : de nouveaux exemples minoritaires, plausibles, qui enrichissent la variété des données.

SMOTE



SMOTE

Avantages

- Génère des exemples **plus diversifiés** que le simple copier-coller.
- Améliore la **capacité de généralisation** du modèle.

Inconvénients

- Peut créer des points **peu réalistes** si les données sont bruitées.
- Risque de **chevauchement des classes** (les synthétiques peuvent s'approcher trop de la classe majoritaire).

```
# Installer si besoin : pip install imbalanced-learn scikit-learn pandas
```

```
from imblearn.over_sampling import SMOTE
from sklearn.datasets import make_classification
from collections import Counter
```

```
# 1) Créons un dataset déséquilibré
```

```
X, y = make_classification(
    n_samples=1000, n_features=10, n_informative=5, n_redundant=2,
    weights=[0.9, 0.1], random_state=42
)
```

```
print("Avant SMOTE :", Counter(y))
```

```
# 2) Appliquer SMOTE
```

```
smote = SMOTE(sampling_strategy=1.0, random_state=42, k_neighbors=5)
X_res, y_res = smote.fit_resample(X, y)
```

```
print("Après SMOTE :", Counter(y_res))
```

```
Avant SMOTE : Counter({np.int64(0): 896, np.int64(1): 104})
```

```
Après SMOTE : Counter({np.int64(0): 896, np.int64(1): 896})
```

SMOTE : Variantes

1. Borderline-SMOTE

- Génère de nouveaux exemples **près de la frontière** entre minorité et majorité.
- Utile car les erreurs de classification se produisent surtout aux frontières.

2. SMOTE + Tomek Links

- Après génération par SMOTE, supprime les **paires ambiguës** (Tomek Links) où un minoritaire et un majoritaire sont trop proches.
- Permet de **nettoyer les chevauchements**.

3. SMOTE + ENN (Edited Nearest Neighbors)

- Combine SMOTE et un **nettoyage par voisins** (suppression des exemples mal classés).
- Rend le dataset **plus propre**, mais parfois trop agressif.

4. ADASYN (Adaptive Synthetic Sampling)

- Génère plus de points **dans les zones difficiles**, là où la minorité est entourée de majoritaires.
- Améliore la détection des cas rares, mais peut introduire du bruit.

5. KMeans-SMOTE

- Applique d'abord un **clustering (KMeans)** sur la minorité, puis génère des points synthétiques **dans chaque cluster**.
- Préserve la **structure interne** de la minorité, plus robuste mais plus coûteux

SMOTE : bonnes pratiques

- **Toujours séparer train/test AVANT l'oversampling** → sinon fuite de données.
- **Combiner SMOTE avec undersampling** → équilibre plus naturel (ex. : SMOTE + Tomek).
- **Utiliser des métriques adaptées** (F1-score, PR-AUC, Recall) plutôt que l'accuracy.
- **Adapter le nombre de voisins (k) :**
 - k trop petit → peu de diversité.
 - k trop grand → risque de générer des points irréalistes.
- **Évaluer plusieurs variantes** (Borderline, ENN, ADASYN) → adaptées selon les données.
- **Visualiser les données** (si possible en 2D avec t-SNE/PCA) pour vérifier que les points synthétiques sont plausibles.

Augmentation de données

Définition: Technique qui consiste à **créer artificiellement de nouvelles données à partir de données existantes**.

Objectif : enrichir le dataset, réduire le surapprentissage (overfitting) et améliorer la généralisation du modèle.

Méthodes selon le type de données

1. Images

- Transformations géométriques : rotation, translation, zoom, recadrage.
- Transformations photométriques : modification de la luminosité, contraste, bruit, couleur.
- Techniques avancées : GANs (génération d'images réalistes), Mixup, CutMix.

2. Texte (NLP)

- Synonymes / paraphrases.
- Traduction automatique aller-retour (*back-translation*).
- Masquage de mots (ex. : style BERT).
- Ajout de bruit (shuffle, suppression de mots).

3. Audio / Signal

- Ajout de bruit blanc.
- Changement de vitesse ou de tonalité.
- Découpage / mixage de segments.
- Spécial EEG/ECG : fenêtrage, time warping, scaling.

4. Tableaux numériques (tabulaires)

- Oversampling (Random Oversampling, SMOTE, ADASYN).
- Perturbation aléatoire des valeurs numériques.
- Génération par modèles (GANs pour tabulaires).

Under vs Over: Quand choisir ?

- Peu de données totales: oversampling/augmentation (préserver l'info).
- Beaucoup de données majoritaires redondantes: undersampling (réduire l'inutile).
- Frontière complexe: SMOTE/ADASYN (densifier la minorité).
- Variables mixtes: SMOTE-NC. Combinaisons fréquentes: SMOTE + TomekLinks (nettoyage des chevauchements).

Stratégies algorithmiques

Au lieu de modifier la distribution des données, on modifie l'OBJECTIF d'entraînement: pondération des classes dans la perte, apprentissage sensible au coût, méthodes d'ensemble équilibrées, et réglage du seuil de décision pour optimiser une métrique/cost métier.

. Modifier les algorithmes (Algorithm-level approaches)

On garde les données telles quelles, mais on **adapte l'apprentissage**.

- **Cost-sensitive learning** :
 - Donner un poids plus fort aux erreurs sur la minorité.
 - Ex. : `class_weight='balanced'` dans SVM, Logistic Regression, Random Forest.
- **Algorithmes spécialisés** :
 - Balanced Random Forest, EasyEnsemble, BalanceCascade.
 - Réseaux de neurones avec **Focal Loss** ou pondération des classes.

Algorithm-level approaches

```
# 3) Logistic Regression avec class_weight
log_reg = LogisticRegression(class_weight='balanced', max_iter=1000, random_state=42)
log_reg.fit(X_train, y_train)
print("\n--- Logistic Regression ---")
print(classification_report(y_test, log_reg.predict(X_test)))

# 4) SVM avec class_weight
svm = SVC(class_weight='balanced', random_state=42)
svm.fit(X_train, y_train)
print("\n--- SVM ---")
print(classification_report(y_test, svm.predict(X_test)))

# 5) Random Forest avec class_weight
rf = RandomForestClassifier(class_weight='balanced', n_estimators=200, random_state=42)
rf.fit(X_train, y_train)
print("\n--- Random Forest ---")
print(classification_report(y_test, rf.predict(X_test)))
```

Approches hybrides

Une solution robuste combine plusieurs leviers:

1. Nettoyage/édition des instances (Tomek Links, Edited NN) pour réduire le chevauchement;
2. Oversampling ciblé (SMOTE/ADASYN, Borderline) pour densifier;
3. Pondération dans la perte (ou algorithme équilibré) pour orienter l'optimisation;
4. Calibration/ajustement de seuil pour aligner la décision avec le coût métier;
5. Validation correcte (Pipeline + CV stratifiée) pour éviter les fuites.

Pièges fréquents

- Appliquer SMOTE avant le split train/test (fuite).
- Oversampling excessif $\Rightarrow$ surapprentissage.
- Ignorer le déséquilibre dans le sampling des mini-batches.
- Utiliser uniquement ROC-AUC lorsque la minorité est ultra-rare; préférer PR-AUC.
- Oublier d'ajuster le seuil après entraînement.

Laboratoire formatif

Laboratoire formatif

1. **Télécharger** le dataset Kaggle *Credit Card Fraud Detection* (creditcard.csv).
<https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud?resource=download>
2. **Prétraiter** les données :
 - Supprimer/traiter les valeurs manquantes et non numériques.
 - Normaliser les variables si nécessaire (StandardScaler).
 - Séparer en *train/test* (70%/30%).
3. **Appliquer des méthodes d'équilibrage des classes** :
 - Undersampling (classe majoritaire).
 - Oversampling aléatoire.
 - SMOTE.
 - ADASYN.

Laboratoire formatif

4. Tester les algorithmes suivants :

- **Naive Bayes (NB).**
- **Decision Tree** (*avec et sans* `class_weight='balanced'`).
- **SVM linéaire** (*avec et sans* `class_weight='balanced'`).

5. Comparer les performances avant et après équilibrage à l'aide des métriques :

- **Accuracy**
- **Precision, Recall, F1-score (classe fraude)**
- **ROC-AUC et PR-AUC**